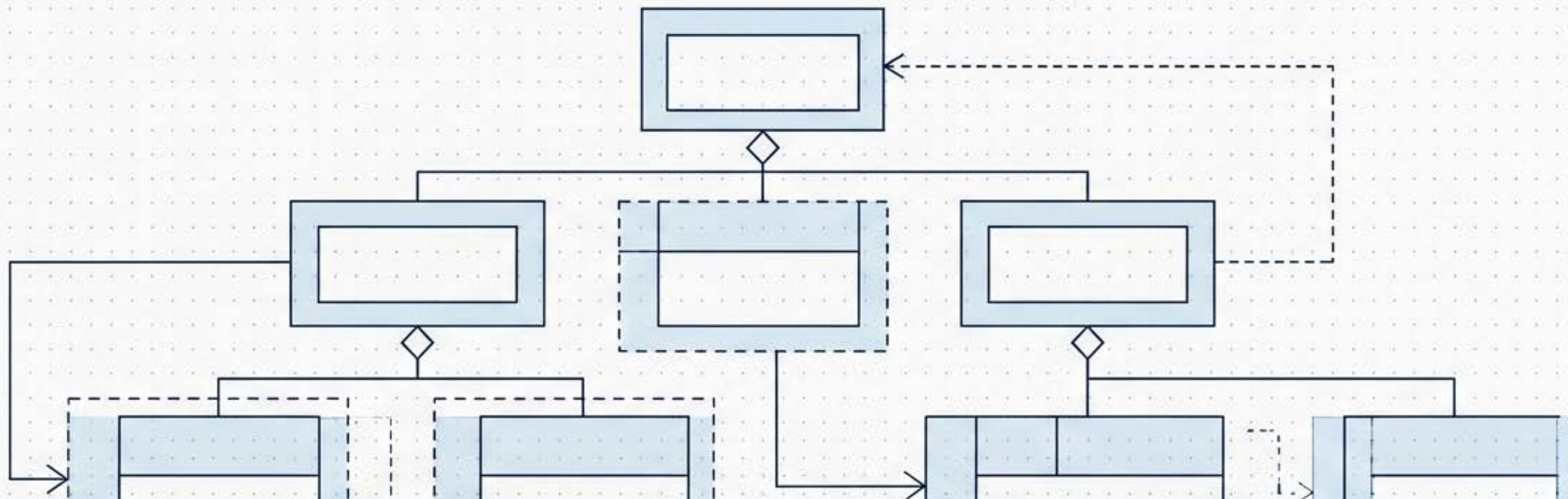




Arquitetura Antes do Código

Dominando Diagramas de Classe UML para a API da Livraria

O Passado: Código Primeiro

```
class Livro {  
  constructor(nome) {  
    this.nome = nome;  
  }  
}
```

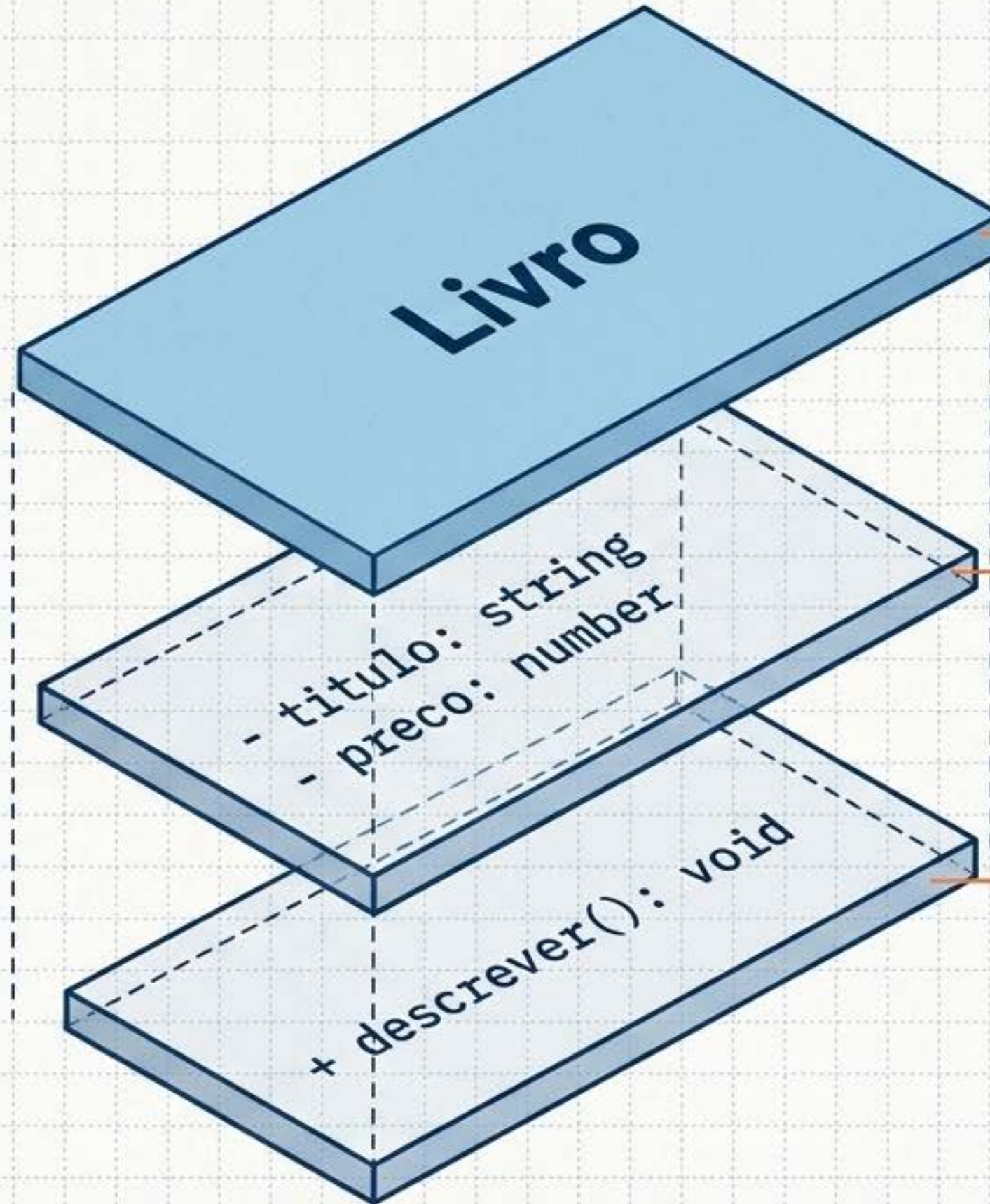
Desde 29/07, escrevemos código para pensar.

Hoje: Desenho Primeiro



Hoje, o processo se inverte. Desenhamos para pensar antes de escrever qualquer linha em JavaScript.

UML é uma linguagem de desenho. Vamos projetar classes que ainda nem existem no código.



Compartmento 1:
Identidade da Classe.

Compartmento 2:
O que a classe SABE
(dados/propriedades).

Compartmento 3:
O que a classe FAZ
(ações/comportamentos).

Rosetta Stone: JavaScript ↔ UML

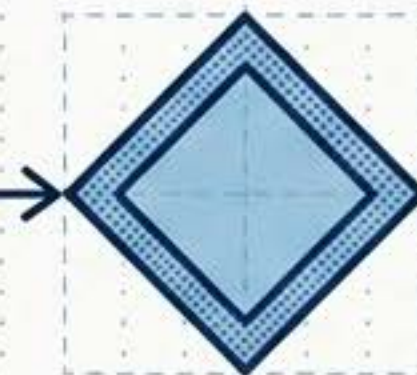
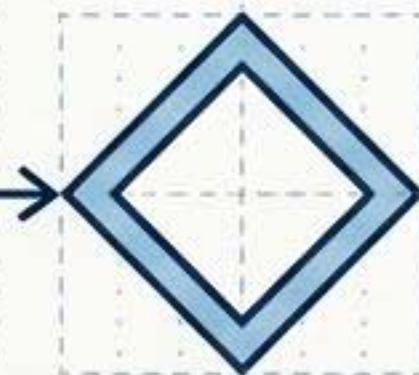
Código JavaScript		Notação UML
Atributo com # (#nome)		 Privado
Atributo ou método sem # (descrever())		 Público
Parâmetro no constructor(nome)		nome: tipo Nome do dado + tipo ao lado
Retorno de função (return valor)		metodo(): tipo metodo(): void

💡 : **void** significa “este método não devolve nada com return, só executa uma ação” (ex: descrever()). Já valorEmEstoque() devolve um número, então é : **number**.



Conexão
Fracã

Conexão
Forte



Associação

Conhece ou usa.

Agregação

Tem um (Fracó).

Composição

Tem um (Forte).

Herança

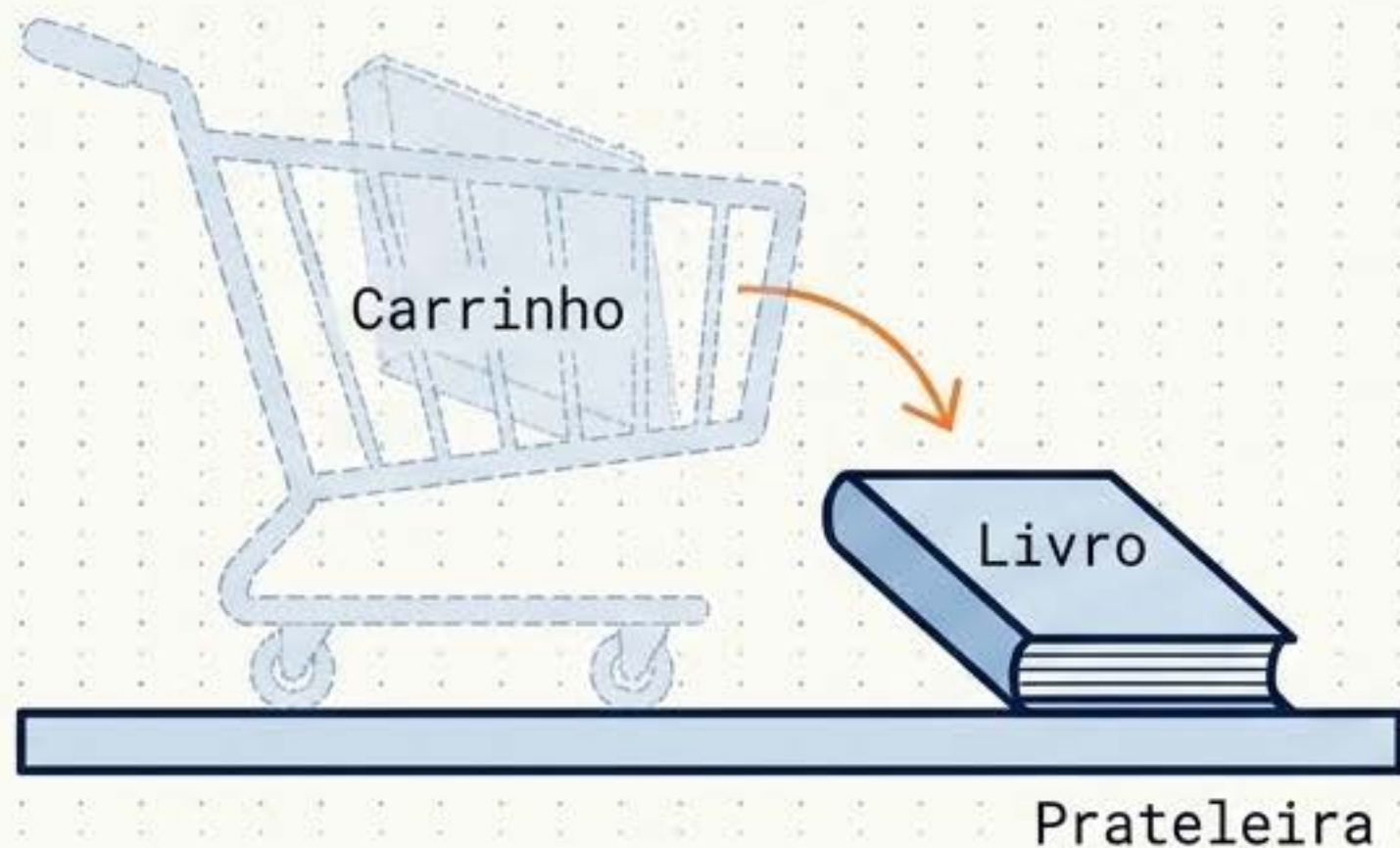
É um
(Dependência total).

Relacionamento	Símbolo na Ponta	Pergunta-Teste	Exemplo da Livraria
Associação		Ele apenas conhece ou usa o outro?	Funcionario → Cliente
Agregação		A parte sobrevive sem o todo? (SIM)	Carrinho ◇→ Livro
Composição		A parte sobrevive sem o todo? (NÃO)	Pedido ◆→ ItemPedido
Herança		É um tipo específico de...?	LivroFisico ← Livro

Atenção: Na Herança, o triângulo vazio sempre aponta para a classe-mãe!

O TESTE DE SOBREVIVÊNCIA: Se eu apagar o TODO, a PARTE sobrevive?

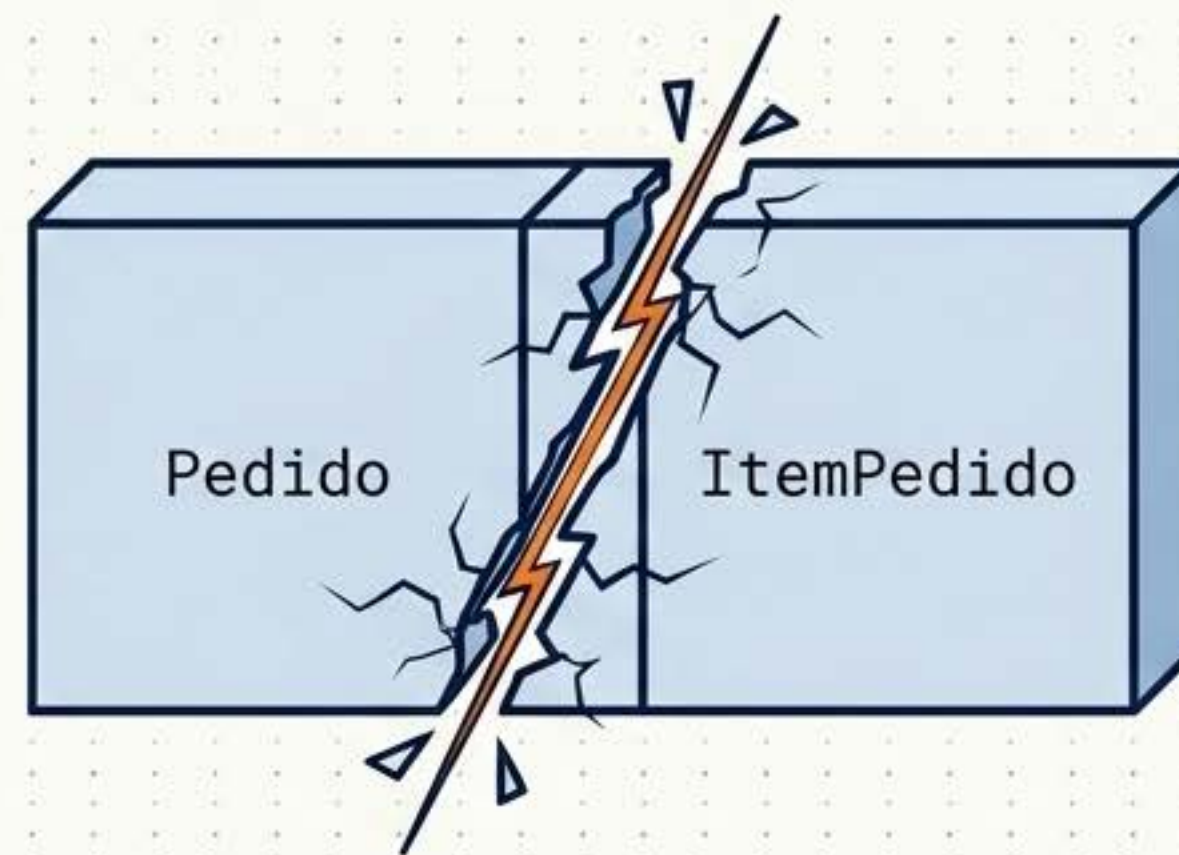
AGREGAÇÃO (Losango Vazio)



SIM. Se apagar o Carrinho, o Livro continua na prateleira. A ligação é "fraca".

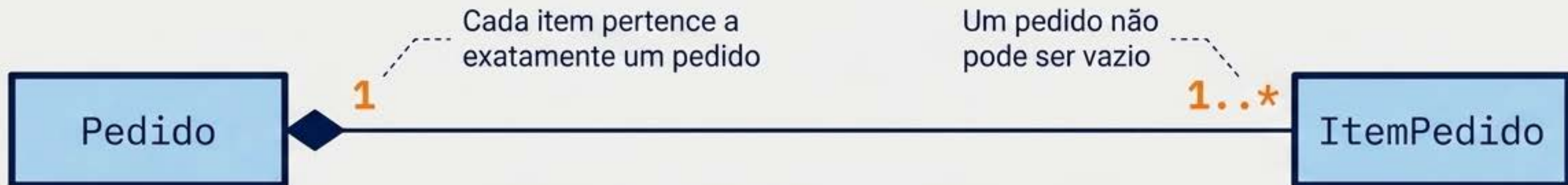
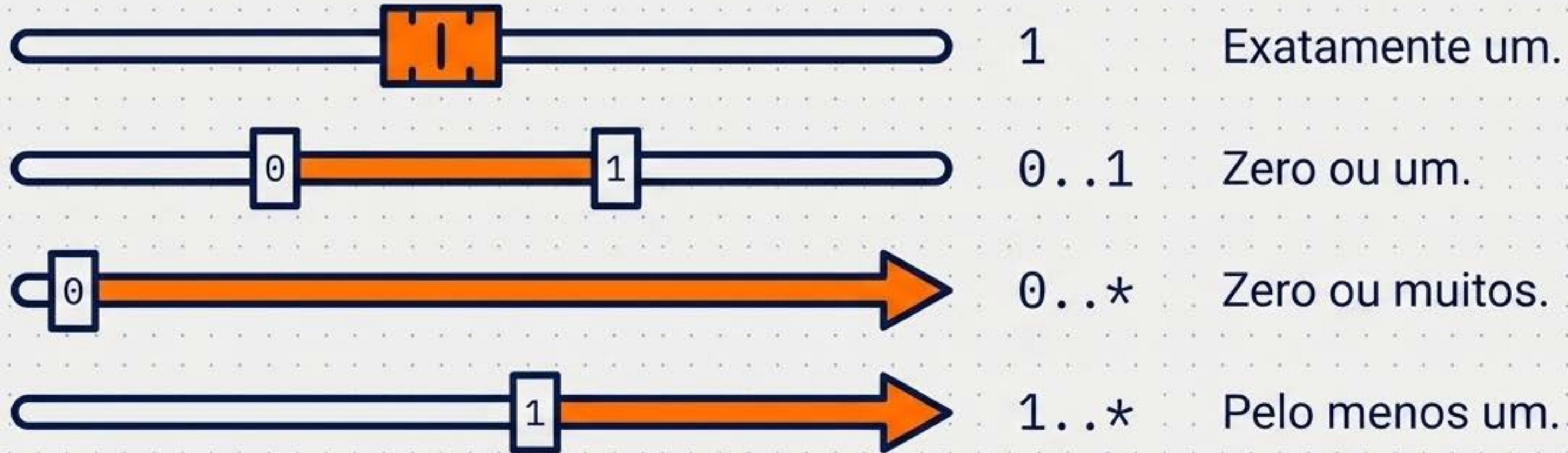
E Livro e Categoria? Se apagar o livro, a categoria "Tecnologia" continua existindo. Também é Agregação!

COMPOSIÇÃO (Losango Preenchido)

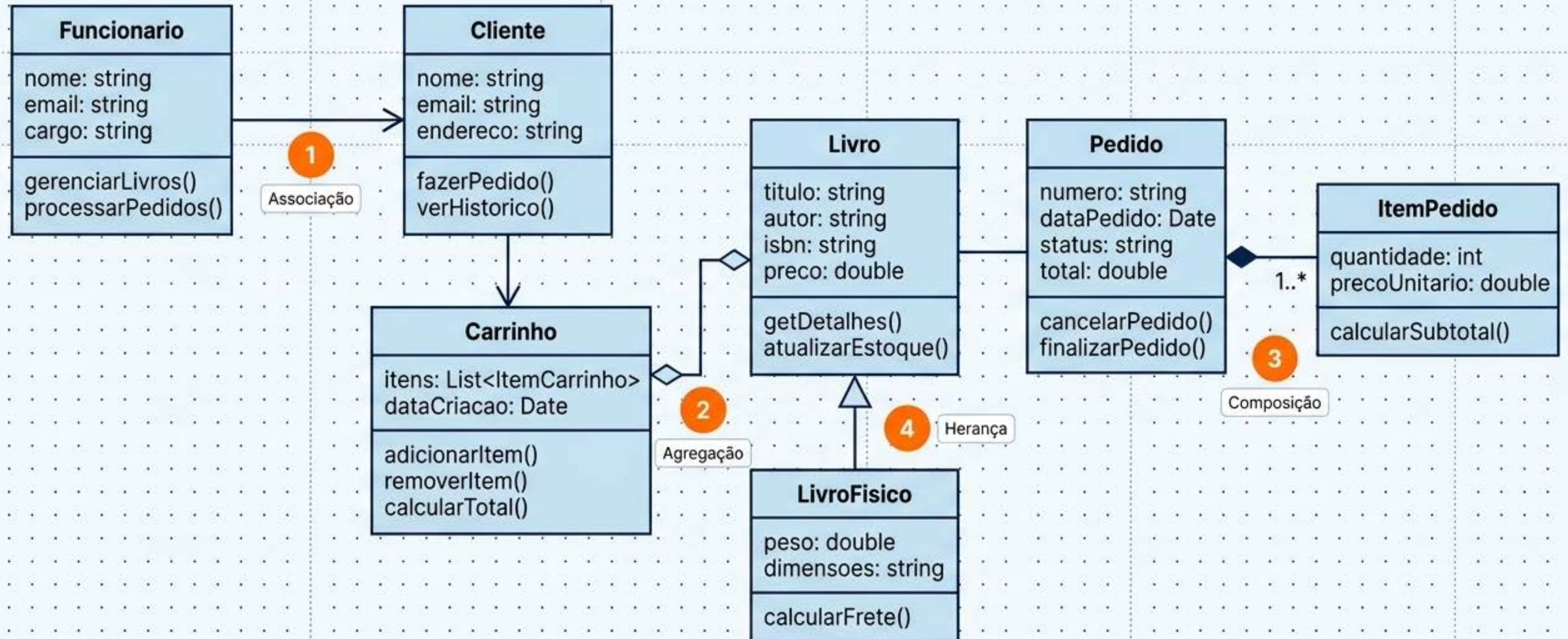


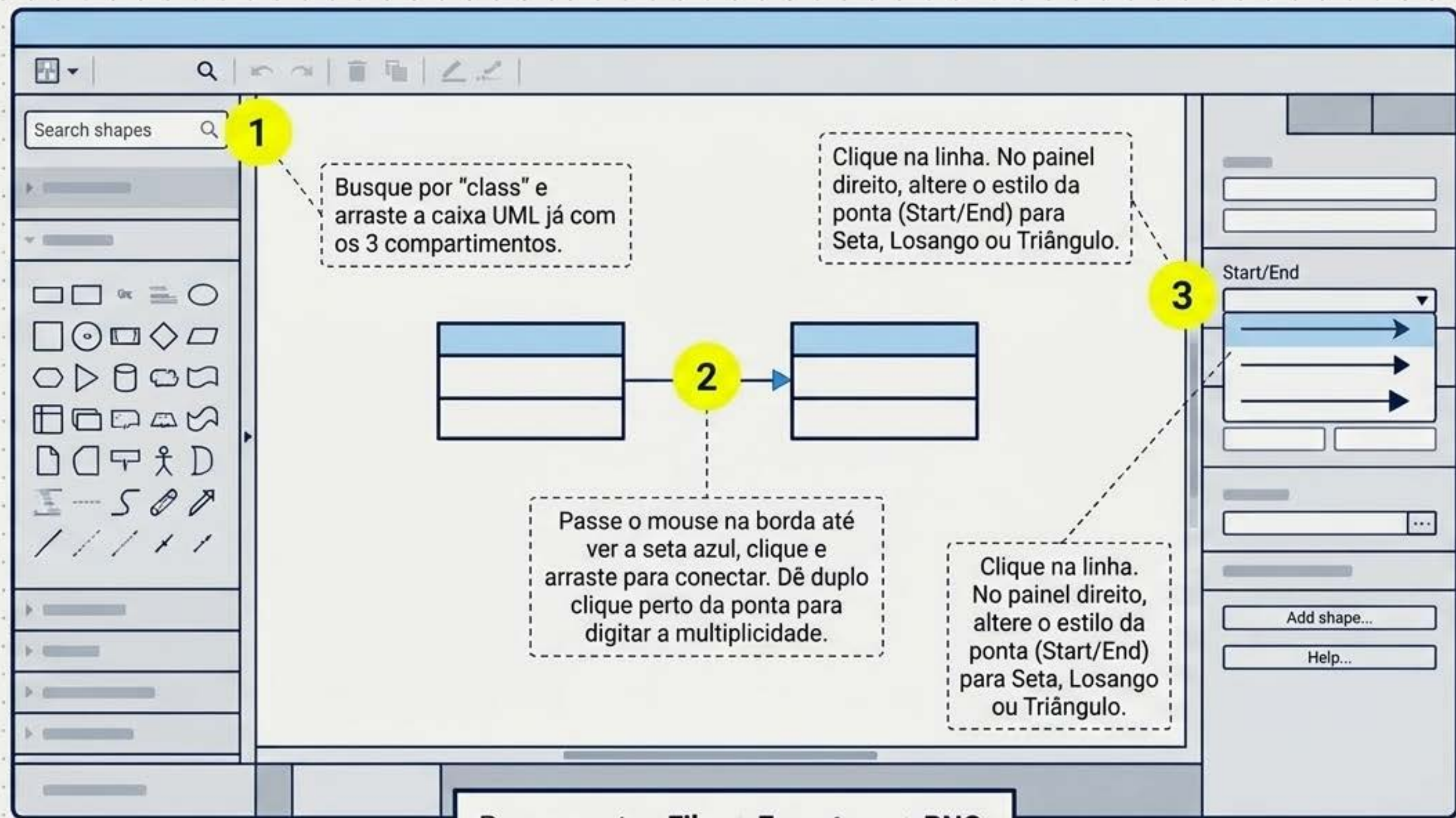
NÃO. Um ItemPedido não faz sentido sozinho sem o seu Pedido. Se apagar o pedido, os itens morrem junto. A ligação é "forte".

Multiplicidade: Quantos de cada lado?



O Mapa Completo

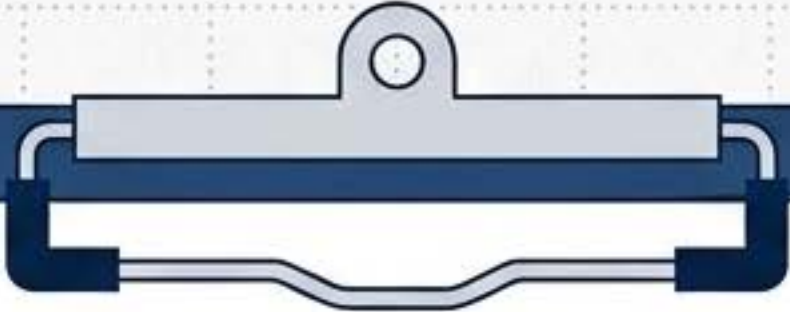




Para exportar: **File** → **Export as** → **PNG**.
Salve seu rascunho com **Ctrl + S**!

Controle de Qualidade: Diagnóstico Comum

Sintoma			Correção
Losango vazio e cheio parecem iguais no desenho.			Confira no painel direito qual ponta da linha está selecionada (start vs end).
Seta de herança apontando para a classe errada.			O triângulo VAZIO sempre aponta para a classe- MÃE (direção invertida).
Confundiu agregação com composição.			Faça a pergunta-teste: “se eu apagar o todo, a parte desaparece?”
Multiplicidade 0..* num caso que exige 1..*.			Pense na regra de negócio (ex: um pedido jamais pode ter zero itens).



Missão de Hoje: Rascunho da API da Livraria

- [] Desenhar caixas de classe com os 3 compartimentos corretos.
- [] Mapear o código existente diferenciando - (privado) de + (público) e indicando os retornos (: void, : tipo).
- [] Criar as novas classes Pedido e ItemPedido e estabelecer a composição.
- [] Configurar as pontas das linhas corretamente (Losangos, Triângulos, Setas).
- [] Adicionar a multiplicidade em todos os relacionamentos.
- [] Exportar o PNG no Draw.io antes do fim da aula.

Lembre-se: É apenas o rascunho inicial. Projetem com calma, refinaremos juntos na próxima aula.